

Chunk-First API Surface Recovery from JavaScript Bundles

J. S. Yang

Independent Researcher

yangjs059@gmail.com

ORCID: 0009-0004-4554-7197

DOI: 10.5281/zenodo.21106612

July 1, 2026

Abstract

Modern single-page applications often place API client code outside the JavaScript files directly visible from the initial HTML response. Lazy route chunks, bundler runtime tables, dynamic imports, and framework manifests can hide endpoint evidence from URL extractors that only scan the entry corpus. This paper presents Chunk-First API Surface Recovery (CF-ASR), a reproducible static-analysis workflow that expands the production JavaScript corpus before endpoint extraction and emits endpoint-oriented records with source provenance, method hints, dynamic path skeletons, and conservative endpoint-only filtering.

Across 15 reviewed production-mode application benchmarks, deterministic CF-ASR recovers strict endpoint paths with precision 0.94 and recall 0.92. A separately scored bounded model-assisted review for composed endpoints raises recall to 0.95. A request-shape study finds visible path, query, body, or header evidence for 251 of 650 review-confirmed recovered endpoint paths, showing that parameter evidence is useful but less complete than endpoint recovery. Recovered records are intended for authorized triage and dynamic testing, not as proof that endpoints are live, reachable, authorized, or vulnerable.

1 Introduction

Production web applications increasingly deliver server-facing behavior through bundled JavaScript. Even without backend source code or API documentation, the client bundle can expose API-like paths, HTTP method hints, feature ownership, authorization flow clues, and lazy-loaded functionality. For authorized penetration testing and defensive review, these signals can help build a candidate endpoint inventory before dynamic replay.

The central problem is that modern frontend deployment is not a flat JavaScript file. A production page may load one entry bundle, while route handlers, admin modules, service clients, and feature-specific API calls live in asynchronously loaded chunks. A URL extractor that only scans the initially visible files can therefore undercount the client-visible API surface. This problem is especially relevant in black-box test-

ing, where the tester may have a deployed web entry point but no Swagger/OpenAPI file, backend source tree, or route inventory.

CF-ASR treats acquisition as part of endpoint recovery. It starts from a web entry point, expands the JavaScript corpus by recovering observable chunk references, beautifies minified bundles, scans the expanded corpus for endpoint evidence, and emits a strict backend/API-oriented candidate inventory. The workflow records source locations and filtering decisions so a human reviewer can inspect why an endpoint was accepted or rejected. The method is intentionally evidence-bounded: it does not validate endpoint liveness, bypass authentication, or infer server-side schemas.

This paper makes four main contributions:

1. It identifies chunk-hidden endpoint evidence as a practical failure mode for web-entry JavaScript reconnaissance.
2. It implements a chunk-first, static-rule-first endpoint recovery workflow with source-line provenance, method hints, dynamic path skeletons, backend-prefix inference, and endpoint-only filtering.
3. It evaluates CF-ASR against URL/path collectors under a web-entry benchmark and reports CF-ASR-specific method, caller, prefix, and request-shape evidence separately from the URL/path comparison.
4. It studies bounded composed-endpoint recovery and request-shape evidence extraction as separately scored analyses, making the source of each reported result explicit.

2 Background and Related Work

Bundlers such as Webpack and modern build systems such as Vite, Next.js, and Nuxt commonly split application code into entry bundles and lazy chunks [1–4]. The initial HTML response may expose only runtime code and a shell UI. The browser later loads feature chunks through runtime loader tables, dynamic imports, route manifests, modulepreload references, or relative ESM imports.

Existing tools such as LinkFinder, JSLuice, Katana, and gospider are valuable reconnaissance baselines, but their out-

puts are generally URL- or crawl-path-oriented [5–8]. They help find strings or reachable links, but they do not normally reconstruct the lazy JavaScript corpus first, distinguish back-end API operations from frontend routes and static assets with a shared endpoint-only policy, or attach each candidate to source-line evidence and extraction rules.

Prior academic work has also shown that client-side JavaScript can reveal server-side endpoints missed by dynamic crawlers [9, 10]. That work is close prior art. This paper therefore does not claim to be the first static JavaScript endpoint-discovery method. The distinction is operational: CF-ASR studies a web-entry, chunk-first workflow for production bundles and evaluates whether the resulting endpoint-only, source-grounded endpoint inventory reduces triage cost for authorized testers.

Semgrep is used as a practical structured extraction engine because request evidence often appears inside object literals, wrapper calls, template strings, method-specific helpers, and generated service maps [11–13]. Babel parser and traversal support local AST context recovery [14, 15]. The design intentionally avoids whole-program data-flow claims; the prototype is a source-grounded endpoint inventory generator rather than a server-route verifier.

3 Method

Figure 2 summarizes the end-to-end workflow. The core principle is ordering: acquire the production JavaScript corpus first, then extract endpoint evidence. Endpoint extraction from the initial corpus alone answers the wrong question when endpoint-bearing code is lazy loaded.

3.1 Running example: from entry URL to endpoint record

A typical CF-ASR run starts from a single authorized web entry point. The HTML page exposes one or a few entry bundles. The entry bundle may contain only the shell, router, and runtime loader, but it can also contain a chunk table or dynamic import expression that names a lazy route chunk. After the chunk is downloaded and beautified, the endpoint scanner may find source-grounded evidence such as a request wrapper call or object-map entry for `/rest/products/search`. The normalized strict record keeps the recovered path, method hint when visible, source file, line number, extraction rule, and filtering reason.

This example illustrates the scope boundary. The output record is useful because it tells an analyst where the client-side evidence is located and why the candidate survived endpoint-only filtering. It does not prove that `/rest/products/search` is live, unauthenticated, or vulnerable. Liveness, authorization, and exploitability remain dynamic testing questions.

Pattern family	Evidence used by acquisition
HTML entry assets	<code>script</code> , <code>modulepreload</code> , and <code>preload</code> references in the initial page
Webpack runtime	Chunk-id tables, filename builders, and lazy route chunk references
Dynamic import	Literal <code>import(...)</code> targets and generated lazy component wrappers
ESM graph	Relative static imports from already captured JavaScript files
Vite/Next/Nuxt	Manifest-like references, dependency lists, and framework chunk naming conventions

Table 1: Chunk-reference pattern families handled by the prototype implementation. The recognizers cover common observable patterns; encrypted strings, arbitrary runtime computation, and authenticated chunk discovery remain outside the evaluated scope.

3.2 Chunk-first acquisition

The acquisition module fetches the HTML entry point, downloads directly referenced JavaScript, extracts observable chunk references, and recursively downloads newly discovered chunks within bounded rounds. Recognizers cover common Webpack-style chunk maps, direct chunk strings, dynamic import literals, static ESM imports, Vite dependency lists, and Next/Nuxt manifest-like references. The output is an expanded, provenance-labeled corpus rather than a claim of complete runtime coverage. Unsupported runtime-only loaders, encrypted strings, service-worker paths, and authenticated chunks remain limitations.

Table 1 summarizes the acquisition patterns without listing every recognizer regular expression. For reproducibility, CF-ASR records which files came from the initial entry and which files were added by recursive chunk discovery. This lets the evaluation separate the effect of chunk acquisition from the effect of endpoint extraction.

3.3 Endpoint extraction and filtering

After beautification, CF-ASR combines Semgrep rules with local JavaScript parsing. Semgrep rules identify direct HTTP calls, request configuration objects, template literals, API object maps, and service-wrapper patterns. Regex and string fallbacks remain useful for low-structure evidence and smoke tests, but structured rules are preferred when broad reusable patterns can be validated.

Endpoint candidates are normalized into path skeletons where possible. For example, `/api/users/$id` and `"/api/users/" + id` become `/api/users/{id}`. Strict endpoint filtering then removes static assets, frontend routes, framework internals, source paths, and weak parent-prefix-only evidence unless additional backend/API evidence exists. This produces a conservative backend-oriented worklist.

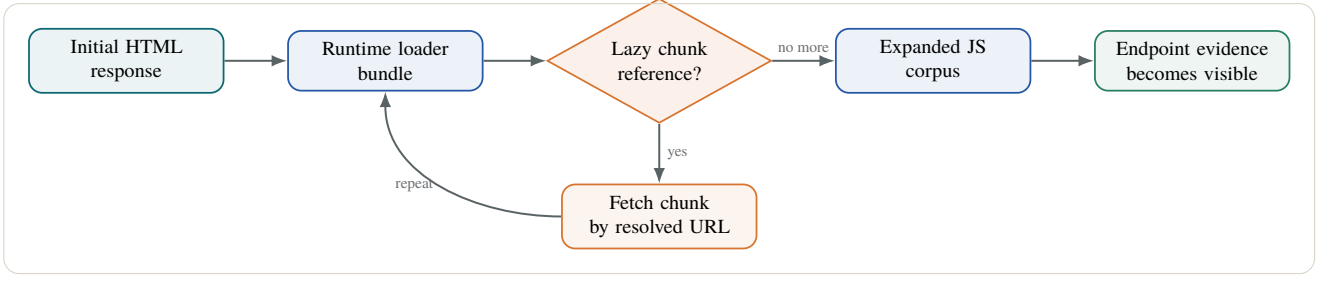


Figure 1: Chunk-hidden endpoint pattern. The endpoint evidence is absent from an entry-only scan, then becomes visible only after the runtime loader reveals and CF-ASR fetches lazy chunks.

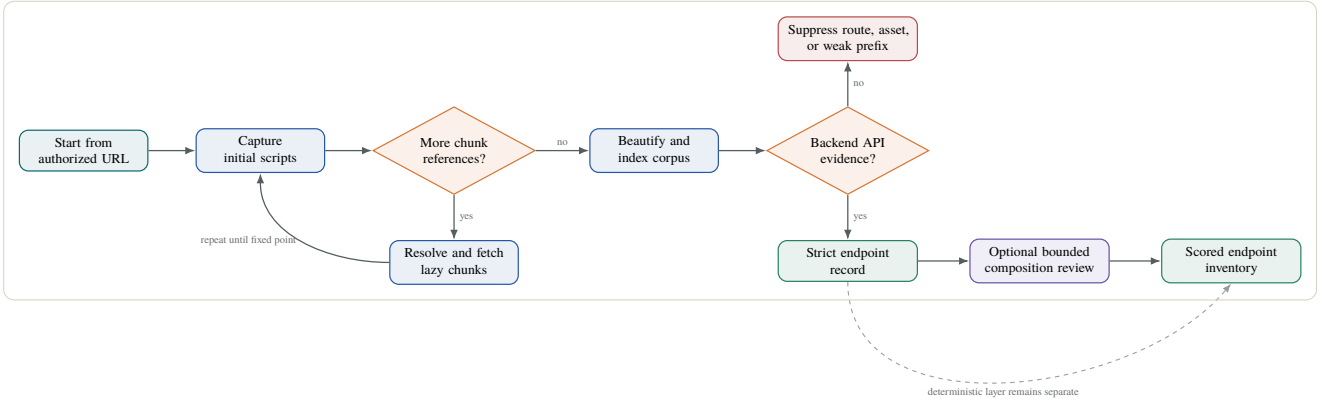


Figure 2: CF-ASR end-to-end recovery workflow. Chunk recovery is iterated to a bounded fixed point before corpus indexing and endpoint evidence extraction. Strict deterministic records remain separate from optional bounded composition review.

Backend-prefix recovery is handled before final endpoint normalization. Many applications define a uniform prefix such as `/api`, `/rest`, a versioned prefix, or a request-client `baseUrl`, then keep service modules as relative paths. CF-ASR records such prefixes when they are source-grounded and composes them with nearby service paths only when the evidence indicates the same request client or wrapper family. If no reliable prefix is found, the prefix is left empty rather than invented.

The AST step is local and anchor-first. Given an endpoint-like string or Semgrep match, CF-ASR uses JavaScript parsing to recover the nearest useful context, such as an enclosing object property, wrapper call, function, class method, or module-level service map. This is why the output can report source/caller evidence rather than only a raw string. The same design also defines the limit: it is not full inter-procedural data flow, and it can miss endpoints whose path is assembled through runtime state or far-away aliases.

Strict filtering uses negative and positive evidence. Static assets, source paths, frontend routes, framework internals, and root-only prefixes such as `/admin` are suppressed unless stronger backend evidence exists. API-like paths such as `/api/system/user/list_all`, `/rest/products/search`, and `/graphql` are retained

when supported by request context, API namespace evidence, or backend-prefix composition. This policy is deliberately conservative because a smaller auditable endpoint inventory is more useful to an authorized tester than a large mixed URL list.

3.4 Bounded model-assisted recovery

The deterministic recovery path consists of chunk-first acquisition, static Semgrep/AST extraction, backend-prefix composition, and strict endpoint filtering. The model-assisted analyses are invoked only after this static pipeline has produced auditable candidates or identified reviewed misses.

The first analysis targets composed endpoint false negatives. The task generator starts from deterministic strict misses, supplies bounded local code context plus target-local true-positive examples, and asks the model to return source-grounded composed-endpoint candidates. The model may request constrained local evidence retrieval, but shell and network access are not exposed. These candidates are scored separately from deterministic strict output.

The model-assisted stage uses a bounded few-shot prompt: reviewed positive examples are provided to calibrate the decision boundary, while each accepted candidate must cite local source evidence. This keeps the model-assisted layer

separate from deterministic recovery and limits unsupported generation.

The second analysis is request-shape enrichment. It operates on already recovered endpoint records and asks for visible path parameters, query keys, body fields, and headers. This problem is harder than endpoint recovery because path strings are often local constants, while request fields can be split across UI state, form models, object spreads, serializers, request wrappers, global interceptors, and backend-only validation schemas. The validator therefore rejects unsupported source lines, unresolved minified variables such as `e` from `post(..., e)`, and wrapper metadata such as `url`, `path`, `method`, `apiPath`, or `payload` when those fields describe transport rather than target API schema. This analysis is an analyst aid, not OpenAPI generation.

The LLM is not given complete bundled applications as a prompt. Production bundles can be too large for reliable context use, and a model-only scan would be harder to reproduce, audit, and constrain. Instead, deterministic extraction first narrows the task to specific candidate endpoints, source windows, and target-local examples. The model may ask for bounded evidence retrieval over the captured and beautified corpus, but accepted answers must cite source-grounded evidence and pass validator checks. This design uses the model for local composition and parameter interpretation, not as an unconstrained endpoint oracle.

4 Experimental Setup

The main benchmark uses 15 reviewed production-mode open-source applications with frontend-visible endpoint labels. Benchmarks are included only when the application can be captured from a deployed web entry point and the reviewed labels describe endpoints visible through the frontend bundle or its recovered chunks. Development-mode frontend source is not used as paper evidence.

Each workflow starts from the same web entry point for a given application. CF-ASR captures the production JavaScript corpus, records input and output artifacts under `outputs/<sample_id>/`, and preserves source locations for accepted endpoint records. Reviewed labels are normalized to endpoint skeletons before scoring so dynamic paths such as `/users/{id}` can be compared consistently.

The comparison scope is narrow by design. Katana, LinkFinder, JSLuice, and gospider are evaluated only on normalized URL/path overlap because they do not emit the same method, caller, backend-prefix, or request-shape evidence as CF-ASR. Those CF-ASR-specific capabilities are reported separately and are not included in the URL/path baseline comparison.

5 Evaluation Protocol

We use the standard counts: true positives are normalized endpoint skeletons matching reviewed labels; false positives are reported endpoint skeletons outside the reviewed set; false neg-

atives are reviewed labels not recovered by the tool. Dynamic skeleton normalization allows explicit CF-ASR records such as `/projects/{id}` to match reviewed dynamic paths when the scoring scope supports skeletons.

Notation in the results tables is compact. TP/FP/FN denotes true positives, false positives, and false negatives. P/R denotes precision and recall; P/R/F1 adds F1 score. Request-shape field tables spell out path parameters, query keys, body fields, and headers rather than using the P/Q/B/H shorthand.

Table 2 maps the compact labels used in paper tables to exact benchmark identifiers. The extended technical report and archived evaluation artifacts retain the full identifiers.

6 Results

Table 3 reports the main CF-ASR strict URL-entry benchmark in a readable form. The result is mixed rather than universal dominance: CF-ASR is strongest on administrative and chunked applications where endpoint-only filtering removes large URL-collector noise, while some targets still expose recall gaps.

Benchmark	Reviewed paths	TP	FP	FN	Precision	Recall
NoteMate	30	30	0	0	1.00	1.00
VueFastAPI	30	30	0	0	1.00	1.00
AEGIS	13	13	0	0	1.00	1.00
Kinit	63	57	0	6	1.00	0.91
JuiceShop	60	44	16	16	0.73	0.73
PyBoiler	16	16	3	0	0.84	1.00
Portal	18	18	0	0	1.00	1.00
FastAPIAdmin	5	5	1	0	0.83	1.00
OpenCloud	54	39	0	15	1.00	0.72
Capital	12	11	1	1	0.92	0.92
Crapibara	35	27	1	8	0.96	0.77
LikeAdmin	128	124	7	4	0.95	0.97
RuoYi-Vue3	100	100	6	0	0.94	1.00
RuoYi-FastAPI	106	104	5	2	0.95	0.98
Angular RealWorld	12	12	0	0	1.00	1.00
Aggregate	682	630	40	52	0.94	0.92

Table 3: Main CF-ASR strict URL-entry benchmark over 15 reviewed production-mode applications. TP, FP, and FN denote true positives, false positives, and false negatives after normalized comparison with reviewed endpoint labels. Short labels are used for layout; exact benchmark identifiers are retained in the archived evaluation artifacts. The three added applications extend endpoint-recovery evidence but should not be read as three independent ecosystems: two are RuoYi-family administrative systems, while Angular RealWorld adds framework diversity.

Table 4 summarizes the best-scoring URL-oriented baseline for each reviewed benchmark application, selected by F1 score from Katana, LinkFinder, JSLuice, and gospider where outputs are available. Full per-tool outputs and row-level details are archived with the artifact.

The acquisition layer is itself a measurable part of the result. Across the 15 reviewed applications, CF-ASR expanded 35 initially visible JavaScript assets to 792 analyzed JavaScript files before endpoint extraction. This is not a standalone endpoint score, but it explains why a web-entry scanner that only

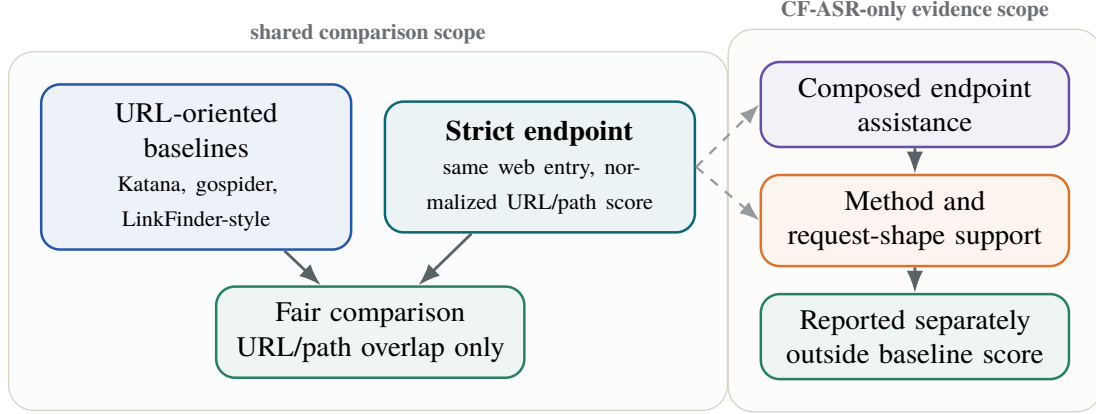


Figure 3: Evaluation-scope boundary. Baseline comparisons use normalized URL/path overlap from the same web entry point. CF-ASR-specific method, request-shape, and composed-endpoint evidence is reported separately because the URL-oriented baselines do not emit those records.

Short label	Exact benchmark identifier	Role in evaluation
NoteMate	notemate_admin_local	Vue/Vite administrative application
VueFastAPI	vue_fastapi_admin_local	Vue/FastAPI administrative application
AEGIS	aegis_local	React/Vite/FastAPI application
Kinit	kinit_local	complex chunked administrative application
JuiceShop	juice_shop_local	known vulnerable benchmark
PyBoiler	python_boilerplate_fastapi_react_local	FastAPI/React application
Portal	client_portal_local	dynamic-skeleton limitation case
FastAPIAdmin	fastapi_react_admin_local	small React/Vite administrative application
OpenCloud	opencloudtouch_local	template-fetch recovery case
Capital	capital_local	high-noise comparison case
Crapibara	crapibara_local	composed-path recovery case
LikeAdmin	likeadmin_python_local	large Vue/Vite administrative application
RuoYi-Vue3	ruoyi_vue3_local	additional RuoYi-family administrative benchmark
RuoYi-FastAPI	ruoyi_vue3_fastapi_local	additional RuoYi-family administrative benchmark
Angular RealWorld	angular_realworld_local	additional Angular application benchmark

Table 2: Mapping between compact table labels and exact benchmark identifiers.

analyzes the initial corpus can miss endpoint evidence that the browser would load later.

Method recovery is reported separately from endpoint-path scoring. Across the same 15 reviewed applications, strict records with explicit HTTP methods produce 732 true positives, 25 false positives, and 102 false negatives at method-path granularity, for precision 0.97 and recall 0.88. The high precision is useful for triage, but the remaining misses show that some applications hide HTTP verbs inside wrappers, defaults, or runtime request clients. The evaluated result is endpoint-surface recovery with method hints, not complete HTTP-method reconstruction.

The two-layer composed-endpoint evaluation raises aggregate recall from 0.92 to 0.95 on the 15-application benchmark, with TP/FP/FN = 650/40/32 combined endpoint-recovery results. These results indicate that model assistance can recover

additional composed endpoints when the captured frontend contains source-grounded evidence, but it should reject candidates when evidence is absent or only pattern-similar.

Tables 5 and 6 report the request-shape enrichment experiment. This is not endpoint TP/FP scoring. It measures whether review-confirmed recovered endpoint paths contain visible request-field evidence. The evaluation uses all 15 reviewed applications after deterministic-plus-assisted endpoint recovery. Known endpoint false positives and broad candidate paths are excluded before request-shape analysis.

These counts measure coverage, not accuracy. They answer: after endpoint recovery has confirmed a path, how often does the captured frontend bundle expose useful request-shape evidence nearby? The aggregate 38.6% endpoint-path coverage provides useful support for analyst review, but it is much weaker than endpoint recovery. This contrast is expected: end-

Benchmark	Best baseline	TP	FP	FN	Base P	Base R	CF-ASR P	CF-ASR R
NoteMate	Katana	30	1	0	0.97	1.00	1.00	1.00
VueFastAPI	Katana	30	4	0	0.88	1.00	1.00	1.00
AEGIS	Katana	13	6	0	0.68	1.00	1.00	1.00
Kinit	gospider	48	109	15	0.31	0.76	1.00	0.91
JuiceShop	Katana	41	22	19	0.65	0.68	0.73	0.73
PyBoiler	Katana	5	3	11	0.63	0.31	0.84	1.00
Portal	LinkFinder	8	11	10	0.42	0.44	1.00	1.00
FastAPIAdmin	LinkFinder	4	14	1	0.22	0.80	0.83	1.00
OpenCloud	Katana	0	8	54	0.00	0.00	1.00	0.72
Capital	LinkFinder	6	77	6	0.07	0.50	0.92	0.92
Crapibara	Katana	5	3	30	0.63	0.14	0.96	0.77
LikeAdmin	Katana	15	21	113	0.42	0.12	0.95	0.97
RuoYi-Vue3	Katana	64	100	36	0.39	0.64	0.94	1.00
RuoYi-FastAPI	Katana	68	129	38	0.35	0.64	0.95	0.98
Angular RealWorld	JSLuice	5	17	7	0.23	0.42	1.00	1.00
Aggregate	Best per application	342	525	340	0.39	0.50	0.94	0.92

Table 4: Comparison against the best-scoring URL-oriented baseline for each reviewed benchmark application. Columns report true positives, false positives, false negatives, precision, and recall after normalized path comparison with reviewed endpoint labels. Full per-tool outputs and row-level interpretation are archived with the artifact.

Scope	Recovered	Static	Enriched	Coverage
Initial 12 apps	432	88	154	35.6%
Added 3 apps	218	71	97	44.5%
All 15 apps	650	159	251	38.6%

Table 5: Endpoint-level request-shape coverage over review-confirmed recovered endpoint paths. Static-supported paths have deterministic request-shape evidence before model-assisted enrichment. Enriched paths have at least one visible path, query, body, or header item after bounded model-assisted review. These counts measure coverage, not parameter precision/recall.

Scope	Path params	Query keys	Body fields	Headers
Initial 12 reviewed apps	73	65	195	60
Added 3 reviewed apps	66	3	49	15
All 15 reviewed apps	139	68	244	75

Table 6: Unique visible request-shape field evidence after bounded model-assisted review. Counts are unique endpoint-field pairs by field category and should not be read as TP/FP/FN accuracy counts.

point paths often survive bundling as string evidence, while parameter names may be implicit in UI components, generic data objects, runtime serializers, global request defaults, or server-side validation code that is absent from the frontend artifact. Counting wrapper metadata such as `url`, `params`, or `payload` as API fields would inflate the result, so CF-ASR records unknowns instead. Per-target diagnostic request-shape rows remain in the archived summaries and illustrate the same boundary conditions.

6.1 Result traceability

Table 7 records the artifact source for the aggregate results reported in this paper. The archived traceability file links each aggregate to exact source paths and raw fields.

7 Discussion

Across the reviewed targets, chunk-first acquisition and endpoint-only filtering reduce URL-list noise while preserving source-grounded endpoint evidence. Method hints, caller context, composed-endpoint review, and request-shape support are reported separately because they are not emitted by the URL-oriented baselines.

Model-assisted review improves recall on some composed endpoints, but the assisted results are bounded, source-grounded, and scored separately from deterministic output.

The residual errors are research directions. Some misses are endpoint-granularity mismatches, such as child/action paths being visible while a reviewed parent collection path is not directly grounded. Some misses are candidate-context failures where same-family true positives exist but candidate-specific evidence does not. Some require reasoning across runtime state, service maps, proxy payloads, or framework-specific loaders. These cases motivate future work on source maps, wrapper-aware data flow, authorized runtime traces, and evidence-guided deterministic rule design.

The model-assisted results provide evidence-bounded assistance rather than model authority. The model is useful when deterministic extraction has already narrowed the task and when local code context can ground a candidate. The review is designed to return no candidate when evidence is absent.

Result family	Reported value	Artifact ID
Strict endpoint recovery	TP/FP/FN=630/40/52; P/R=0.94/0.92	A1
Best URL-oriented baseline	TP/FP/FN=342/525/340; P/R=0.39/0.50	A2
Deterministic-plus-assisted endpoint recovery	TP/FP/FN=650/40/32; P/R=0.94/0.95	A1
Chunk acquisition	35 initial JS assets → 792 analyzed JS assets	A3
Method-evidence score	TP/FP/FN=732/25/102; P/R=0.97/0.88	A4
Request-shape review units	808 endpoint-method/call-site units	A5
Request-shape static-supported paths	159/650 recovered paths	A6
Request-shape endpoint coverage	251/650 recovered paths	A5
Request-shape unique field counts	Table 6	A5

Table 7: Result traceability map. Artifact IDs resolve to exact source paths and raw fields in `outputs/paper_main15_traceability.json`: A1 endpoint results, A2 baseline summary, A3 acquisition evidence, A4 method summary, A5 request-shape summary, and A6 static request-shape summary. Precision and recall are rounded to two decimals in the paper; exact values are retained in the artifact.

Case	Observed weakness	Interpretation
JuiceShop	Parent/root and child/action granularity mismatches	Some concrete action paths are source-grounded while a reviewed parent collection path is not directly visible. Accepting the parent automatically would inflate recall but weaken evidence quality.
LikeAdmin	Strong endpoint recovery but weak request-shape recovery	Service wrappers expose many endpoint paths, but nearby context often contains wrapper metadata or generic payload variables rather than concrete query/body/header fields. Counting wrapper keys would overstate parameter recovery.
Crapibara	Object-map and composed-path cases	Endpoint strings can be present in generated maps while method or caller context is separated. This motivates bounded model-assisted review and broader structured patterns, not one-off target rules.

Table 8: Representative residual errors from reviewed benchmark applications. These rows show why deterministic endpoint recovery, composed-endpoint review, and request-shape enrichment are reported as separate measurements.

That behavior matters for security research: a high-recall but hallucination-prone endpoint list would be less useful than a lower-recall inventory whose evidence can be audited.

8 Threats to Validity and Limitations

Internal validity depends on reviewed labels, normalization, and consistent artifact generation. The benchmark normalizes dynamic skeletons so that `/users/{id}` and equivalent source-grounded forms are compared fairly, but labeling mistakes or overly broad normalization could still affect TP/FP/FN counts. The artifact therefore keeps per-target outputs and traceability tables rather than only aggregate numbers.

Construct validity depends on what is being measured. The main benchmark measures path/URL overlap from the same web entry point because the comparison tools are URL-oriented. Baseline tools are not expected to emit method hints, caller context, backend-prefix composition, or request-shape evidence. Those CF-ASR-specific capabilities are reported separately.

External validity is limited by the reviewed production-mode applications. They are more reproducible than ad hoc

live scans, but they do not cover every frontend framework, obfuscator, authentication flow, or deployment style. The three added reviewed applications strengthen the endpoint-recovery and request-shape evidence, but two are RuoYi-family Vue/Vite administrative systems and should not be interpreted as independent ecosystem evidence. The current result supports chunk-first recovery on common modern bundle patterns, not universal coverage.

Reproducibility depends on archiving the code, labels, generated outputs, and traceability metadata used for the reported numbers. The archived artifact contains the generated tables, rule files, hash manifests, and per-target derived evidence needed for independent inspection, while raw local captures and authoring workflow files are excluded.

8.1 Operational limitations

CF-ASR is static and evidence-bounded. It can miss chunks produced only by runtime computation, authentication-gated flows, service workers, encrypted strings, or unsupported loaders. Minified identifiers often lack semantic meaning. The AST context method is local and does not implement full inter-

procedural data flow. Method recovery remains weaker than path recovery when method evidence is hidden in wrappers. Model-assisted analyses are separately scored and should not be merged automatically into strict deterministic output. The evaluation does not attempt complete request-parameter or OpenAPI schema recovery because many facts are runtime- or server-side-only. In the 15-application recovered-endpoint run, request-shape evidence covers 251 of 650 recovered endpoint paths, so it is useful analyst support but not an endpoint-path accuracy metric. The lower request-shape coverage is therefore a measured limitation of frontend-only static evidence, not an endpoint-recovery failure.

The evaluation uses production-mode applications and reviewed frontend-visible labels. This is stronger than unreviewed live crawling, but it is still not a universal measurement of all web applications. Adding more applications without reliable labels would make the benchmark larger but not necessarily more credible. Future work should prioritize additional reviewed production-mode applications, source-map-aware recovery, wrapper-aware local data flow, UI-form-to-request linking, typed client and schema artifact ingestion when available, and controlled dynamic traces that can confirm runtime behavior without turning the static artifact into an exploit tool.

9 Ethics and Responsible Use

This work is intended for authorized testing, defensive review, and reproducible research. The evaluation uses local open-source applications, synthetic fixtures, and metadata-only derived evidence for third-party applications unless redistribution is clearly appropriate. Recovered endpoints should be treated as triage clues for later authorized dynamic testing, not as proof of vulnerability or access.

The paper does not publish live third-party target scans as benchmark truth. Evaluation benchmarks are built from open-source applications, and raw third-party JavaScript is not redistributed unless licensing permits it. If the workflow is applied to a real organization, recovered endpoint records should be handled as potentially sensitive internal reconnaissance data.

10 Artifact and Reproducibility

The Zenodo record is intended to contain this PDF and a compressed reproducibility artifact. The reproducibility artifact contains the CF-ASR prototype scripts needed for fixture reruns and deterministic one-corpus use, reviewed labels, input hash manifests, generated result tables, the main traceability JSON, chunk-acquisition manifests, and per-target derived evidence files. It intentionally excludes raw local captures, `target_work/`, credentials, manuscript source, local review notes, and authoring workflow files.

The intended release license is Creative Commons Attribution 4.0 International (CC BY 4.0) for the paper text, figures, documentation, and project-authored data artifacts. Project-authored code is intended to remain under the MIT License.

Raw third-party JavaScript bundles are not relicensed by this project and should remain metadata-only or separately governed by their upstream licenses.

This paper summarizes the method, benchmark design, and aggregate results. The archived artifact provides the public reproducibility record, including prototype source code, reviewed labels, generated outputs, traceability metadata, and per-target derived evidence files.

11 Conclusion

Chunk-hidden endpoint evidence is a practical failure mode for modern web-entry reconnaissance. CF-ASR addresses this by acquiring lazy JavaScript before endpoint extraction and by emitting endpoint-oriented records with source evidence and conservative filtering. The evaluated results support CF-ASR as a practical API-surface recovery workflow for authorized security review, while the residual errors define clear limits for future work.

References

- [1] “Webpack code splitting guide.” <https://webpack.js.org/guides/code-splitting/>. Official documentation. Accessed 2026-06-27.
- [2] “Vite build options.” <https://vite.dev/config/build-options>. Official documentation. Accessed 2026-06-27.
- [3] “Next.js lazy loading guide.” <https://nextjs.org/docs/app/guides/lazy-loading>. Official documentation. Accessed 2026-06-27.
- [4] “Nuxt components directory documentation.” <https://nuxt.com/docs/4.x/directory-structure/app/components>. Official documentation. Accessed 2026-06-27.
- [5] “Linkfinder.” <https://github.com/GerbenJavado/LinkFinder>. Primary repository. Accessed 2026-06-27.
- [6] “Jsluice.” <https://github.com/BishopFox/jsluice>. Primary repository. Accessed 2026-06-27.
- [7] “Katana repository.” <https://github.com/projectdiscovery/katana>. Primary repository. Accessed 2026-06-27.
- [8] “Gospider.” <https://github.com/jaeles-project/gospider>. Primary repository. Accessed 2026-06-27.
- [9] D. Sigalov and D. Gamayunov, “Finding server-side endpoints with static analysis of client-side javascript,” in *Computer Security. ESORICS 2023 International Workshops*, vol. 14399 of *Lecture Notes in Computer Science*,

pp. 442–458, Springer, 2024. Publisher page. Accessed 2026-06-27.

- [10] D. Sigalov, “Mining http requests from client-side js with static analysis.” <https://blog.secsem.ru/en/mining-requests-from-js-with-static-analysis/>, 2024. Security Lab technical blog. Accessed 2026-06-27.
- [11] “Semgrep.” <https://github.com/semgrep/semgrep>. Primary repository. Accessed 2026-06-27.
- [12] “Semgrep rule syntax documentation.” <https://docs.semgrep.dev/writing-rules/rule-syntax>. Official documentation. Accessed 2026-06-27.
- [13] “Semgrep pattern syntax documentation.” <https://docs.semgrep.dev/writing-rules/pattern-syntax>. Official documentation. Accessed 2026-06-27.
- [14] “Babel parser documentation.” <https://babeljs.io/docs/babel-parser/>. Official documentation. Accessed 2026-06-27.
- [15] “Babel traverse documentation.” <https://babeljs.io/docs/babel-traverse/>. Official documentation. Accessed 2026-06-27.